

JobCopilot

AI 求职助手 — 项目设计文档

基于 LangChain Agent + RAG + Prompt 工程的全栈 AI 应用

文档版本: v1.0

日期: 2026-07-09

作者: 萧仁科

目录

1. 项目概述
2. 技术架构
3. 模块详解
 - 3.1 Prompt 模板模块
 - 3.2 RAG 向量知识库
 - 3.3 LangChain Agent 引擎
 - 3.4 FastAPI 后端
 - 3.5 Vue 3 前端
4. AI 开发流程与规范
5. 部署与使用
6. 项目总结

1. 项目概述

JobCopilot 是一个全栈 AI 求职助手系统。在求职过程中，求职者需要面对大量职位 JD、反复修改简历、撰写个性化求职信、追踪投递状态等繁琐工作。JobCopilot 利用大语言模型 (LLM) 的能力，将整个求职流程自动化、智能化。

1.1 核心功能

- JD 智能分析：使用 Few-shot Prompting 技术，从职位描述中结构化提取技能要求、经验年限、公司文化等关键信息
- 简历匹配与优化：结合 RAG(检索增强生成)技术，评估简历与目标职位的匹配度，并给出定向优化建议，保持真实不虚构
- 求职信生成：采用 Chain-of-Thought 链式 Prompt，依次分析公司需求→匹配个人经历→生成定制化求职信，支持正式商务/亲和自然/技术极客三种风格
- 投递进度管理：SQLite 持久化存储，支持投递全流程状态追踪（待投递→已投递→初筛中→面试中→已发 Offer/已拒绝）
- Agent 自动模式：基于 ReAct 模式(Thought→Action→Observation 循环)，Agent 可以自动规划并执行多步骤求职任务

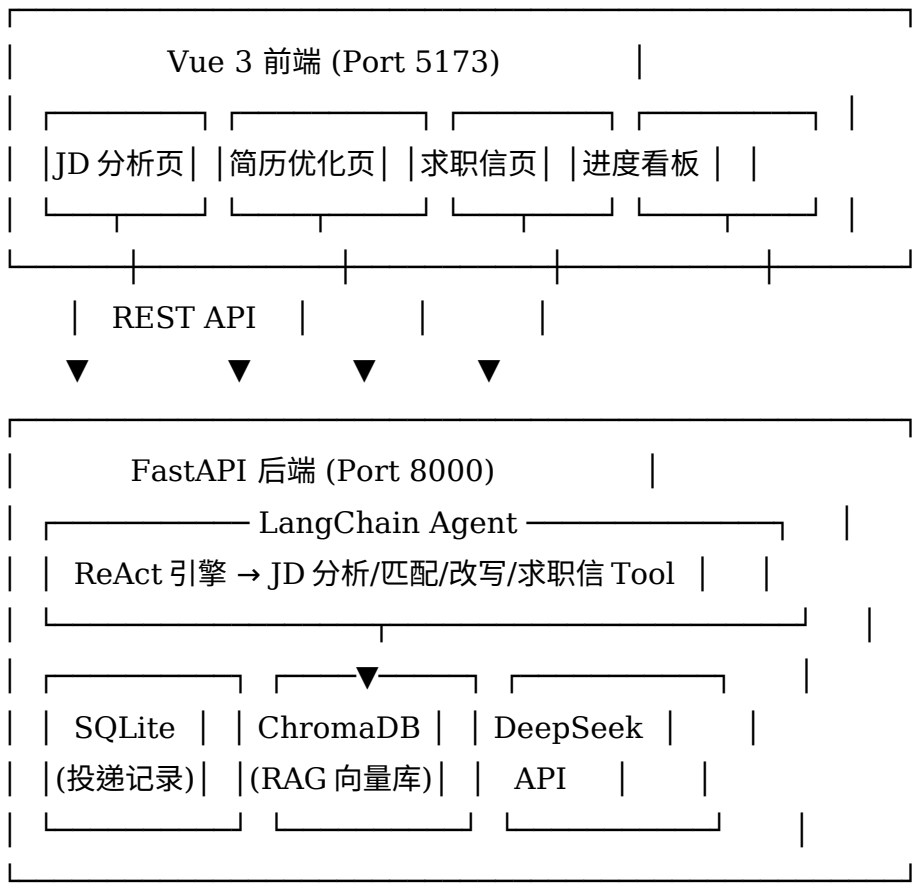
1.2 技术选型理由

技术	选型	理由
LLM 框架	LangChain	生态成熟，ReAct Agent、Prompt 模板、RAG 组件开箱即用
Agent 模式	ReAct Agent	Thought→Action→Observation 循环，适合多步骤任务规划
RAG 向量库	ChromaDB	轻量级，本地持久化，无需 Docker 或外部服务
嵌入模型	BGE-small-zh-v1.5	中文语义搜索专用，本地 CPU 运行，无需 API
Prompt 技术	Few-shot + CoT	Few-shot 引导输出格式，Chain-of-Thought 分解复杂任务
后端	FastAPI	异步支持好，自动生成 Swagger 文档，与 AutoGEO

		经验一致
数据库	SQLite	零配置，适合个人项目，SQLAlchemy ORM 操作
前端	Vue 3 + Element Plus	选型与 AutoGEO 一致，复用技术栈经验

2. 技术架构

2.1 系统架构图(文字版)



2.2 数据流（以简历优化为例）

1. 用户选择JD文本 → 前端发送到 POST /api/resume/match
2. 后端接收 → ChromaDB 检索JD关键词 + 简历片段(RAG) → 构建增强 Prompt
3. Prompt = 系统提示词 + JD要求 + 简历原文 + 检索上下文 → DeepSeek API 调用
4. LLM 返回匹配度评分(0-100) + 逐项分析(技能/经验/学历/素质) + 优化建议
5. 前端展示雷达图分解 + 差距点高亮 + 逐条优化建议
6. 用户点击"定向优化" → 后端调用简历改写 Prompt → 保留真实性前提下改写表述
7. 返回优化后简历(改动标注) → 用户审阅 → 保存到投递记录

3. 模块详解

3.1 Prompt 模板模块 (backend/prompts/)

设计思路

Prompt 工程是 LLM 应用的核心。JobCopilot 中的 Prompt 模板遵循以下设计原则：

- 结构化输出：使用 JSON Schema 约束 LLM 输出格式，确保后处理可靠性
- Few-shot 示例：在 System Prompt 中提供输入-输出示例,引导模型行为
- Chain-of-Thought：复杂任务(如求职信)分解为多个链式步骤,逐步推理
- 角色设定：每类 Prompt 设定专业角色(HR 专家/简历顾问/商业分析师)

三个核心 Prompt 模板

1. JD 分析器 (jd_analyzer.py)

- System Prompt: 设定为"资深 HR 和技术招聘专家"
- 技术: Few-shot Prompting，提供一个完整的 JD→JSON 分析示例
- 输出: position_title, level, hard_skills, bonus_skills, soft_skills,

core_responsibilities 等结构化字段

- Temperature: 0.3 (低温度确保输出稳定)

2. 简历匹配器 (resume_tailor.py)

- System Prompt: 设定为"简历优化专家和招聘顾问"
- 技术: Chain-of-Thought + RAG 检索增强
- CoT 步骤: (1)JD 核心能力分析 (2)匹配关联经历 (3)识别缺失能力 (4)重新表述建议
- 核心原则: 保持诚实，不要虚构经历和技能

3. 求职信生成器 (cover_letter.py)

- 三链式 Prompt Chain:
 - 链 1: 公司需求分析 → 提取公司痛点和理想候选人画像
 - 链 2: 个人经历匹配 → 选出 3 个最能打动面试官的核心卖点
 - 链 3: 求职信生成 → 结合前两链结果生成最终求职信
- 支持三种风格: 正式商务(formal)、亲和自然(casual)、技术极客(tech)
- 每种风格有独立的语调和结构指南

3.2 RAG 向量知识库 (backend/rag/)

设计思路

RAG (Retrieval-Augmented Generation) 在 JobCopilot 中的核心作用是：在 LLM 处理简历匹配和求职信生成时，从向量库中检索与当前 JD 最相关的简历片段和历史职位数据，作为额外上下文注入 Prompt，使生成的回答更加精准和个性化。

技术实现

- 向量数据库: ChromaDB，本地持久化，无需外部服务
- 嵌入模型: BAAI/bge-small-zh-v1.5，中文语义搜索专用，CPU 运行
- 文本分块: RecursiveCharacterTextSplitter，chunk_size=500，overlap=50
- 两个 Collection: "resumes"(简历库) 和 "jds"(职位库)
- 检索策略: similarity_search，简单高效
- 单例模式: 全局唯一的 VectorStore 实例，避免重复加载模型

为什么选择 ChromaDB 而非其他向量库:

- FAISS: 功能强大但依赖复杂，不适合轻量项目
- Pinecone/Weaviate: 需要云服务，增加外部依赖
- pgvector: 需要 PostgreSQL，安装配置重

→ ChromaDB: pip install 即用，基于 SQLite，和我们的整体技术栈高度匹配

3.3 LangChain Agent 引擎 (backend/agent/)

ReAct 模式

ReAct (Reasoning + Acting) 是目前 AI Agent 领域最主流的设计模式。它的核心循环是:

Question (用户问题)

- Thought (思考: 我需要什么工具)
- Action (行动: 调用工具)
- Action Input (输入: 工具参数)
- Observation (观察: 工具返回结果)
- Thought (思考: 还需要什么)
- ... (循环)
- Final Answer (最终回答)

参考了 how-claude-code-works 中 Agent Loop 的设计理念，JobCopilot 的 Agent 引擎也是"上下文组装 → 模型决策 → 工具执行 → 结果注入 → 继续/停止"的循环。Agent 被限定了 8 次最大迭代次数，防止无限循环消耗 API 费用。

四个 Agent 工具

工具名称	功能	输入	关键技术
analyze_jd	JD 结构化分析	JD 全文	Few-shot + JSON Schema
match_resume	简历匹配度评分	JD 分析+简历全文	RAG 检索 + CoT
tailor_resume	简历定向改写	JD 分析+匹配结果+简历	Prompt 模板
generate_cover_letter	求职信生成	JD+简历+姓名+风格	Chain Prompt

双重调用模式

JobCopilot 提供了两种调用方式:

1. Agent 模式 (POST /api/agent/run)
- 用户输入自由文本 → ReAct Agent 自动规划多步骤 → 依次调用工具 → 返回最终结果
 - 例如: "分析这个 JD 然后帮我优化简历并生成求职信" → Agent 自动判断需要 3 个工具依次执行
2. 直接调用模式 (各独立 API 端点)
- 前端直接调用具体的 LLM 服务函数，绕过 Agent 推理环节
 - 特点: 更快、更稳定、成本更低
 - 适合前端按步骤引导用户的场景

3.4 FastAPI 后端 (backend/api/ + main.py)

后端采用 FastAPI 框架，提供 6 个 REST API 端点:

端点	方法	功能	涉及 AI 技术
/api/jd/analyze	POST	JD 结构化分析	Prompt(Few-shot)
/api/resume/match	POST	简历匹配度评分	RAG + Prompt(CoT)
/api/resume/tailor	POST	简历定向改写	Prompt 模板
/api/cover-letter/generate	POST	求职信生成	Prompt Chain
/api/tracker/*	CRUD	投递进度管理	SQLite CRUD

/api/agent/run	POST	Agent 全流程	ReAct Agent
----------------	------	-----------	-------------

数据模型 (SQLAlchemy ORM):

- Application: 投递记录(id, company_name, position_title, jd_text, jd_analysis, match_score, tailored_resume, cover_letter, status, notes, created_at, updated_at)
- Resume: 简历存储(id, name, content, is_active, created_at, updated_at)

3.5 Vue 3 前端 (frontend/)

前端采用 Vue 3 + TypeScript + Element Plus + Pinia 技术栈，与 AutoGEO 项目一致。

五个核心页面

页面	路由	核心组件	功能
仪表盘	/	功能卡片 + 统计面板	项目总览、投递统计、快速导航
JD 分析	/jd	文本输入 + 结果展示	粘贴JD → Few-shot Prompt → 结构化输出
简历优化	/resume	双面板 + 标签切换	匹配度雷达图 + 差距分析 + 改写建议
求职信	/cover-letter	表单 + 预览	三风格选择 + Chain Prompt 生成
投递管理	/tracker	表格 + 对话框	CRUD + 状态流转 + 筛选

4. AI 开发流程与规范

正规的 AI 软件开发项目与普通软件项目有显著区别。以下是建议遵循的开发流程：

4.1 需求分析与设计阶段

1. 需求定义: 明确要解决的业务问题(Prompt 工程 → RAG → Agent 逐层拆解)
2. 技术选型: LLM 选型(DeepSeek vs OpenAI)、框架选型(LangChain vs 自研)、向量库选型
3. 架构设计: 绘制系统架构图、数据流图、API 设计
4. Prompt 设计: 这是 AI 项目区别于传统项目的最核心步骤
 - 手写 System Prompt 和 User Template
 - Few-shot 示例的设计和验证
 - Chain-of-Thought 步骤的拆解
 - JSON Schema 输出格式的定义

4.2 开发实施阶段

AI 项目的开发顺序有讲究，建议遵循"从内到外"原则：

第一步: Prompt 模板 → 在 Playground 中反复测试 Prompt 质量

第二步: RAG 模块 → 文档加载、分块策略、检索验证

第三步: Agent 引擎 → 集成 Prompt 和 RAG，构建 ReAct 循环

第四步: API 接口 → 包装为 FastAPI 端点

第五步: 前端页面 → 最后才做 UI

4.3 代码审查(Code Review)

代码审查是保证项目质量的关键环节。对于 AI 项目，需要特别关注以下方面：

1. Prompt 模板审查

- 检查是否存在 Prompt 注入风险（用户输入直接拼接到 System Prompt）
- 验证 Few-shot 示例的准确性（不要有错误引导）
- 确认 Temperature 设置合理（分析类低至 0.3，创意类高至 0.7-0.9）
- JSON Schema 输出的容错处理（LLM 可能输出非标准 JSON）

2. RAG 模块审查

- 分块策略是否合理（chunk_size 和 overlap 的选择）
- 检索 top_k 值的设定（太少信息不足，太多噪声干扰）
- Embedding 模型选择是否匹配语言（中文不能用 OpenAI 的英文模型）

- 是否有去重和缓存机制

3. Agent 引擎审查

- max_iterations 必须设置上限（防止无限循环消耗 API 费用）
- 错误处理是否完备（API 超时、JSON 解析失败、工具调用异常）
- 是否有成本控制（token 计数、单次预算上限）

4. 安全性审查

- API Key 不能硬编码（应用环境变量或.env 文件）
- .gitignore 必须排除 .env、chroma_db/、*.db
- CORS 配置不能使用通配符*（生产环境）
- 用户输入的 SQL 注入防护（但 SQLAlchemy ORM 已有基础防护）

5. 性能审查

- LLM 调用是否有超时设置（避免用户无限等待）
- Embedding 模型是否合理缓存（避免每次请求重新加载）
- 数据库索引是否到位（频繁查询的字段如 status）
- 前端是否有 loading 和 error 状态展示

4.4 测试规范

AI 项目的测试比传统项目更复杂，因为 LLM 的输出具有不确定性：

1. Prompt 测试: 同一个 Prompt 输入至少测试 3-5 次，观察输出稳定性
2. 边界测试: 空输入、超长输入、特殊字符、不同语言
3. 回归测试: Prompt 修改后重新跑全套测试用例
4. Mock 测试: 对 LLM API 进行 Mock，测试业务逻辑层的正确性
5. 手动审查: AI 生成的简历和求职信需要人工 review（这是 AI 项目特有的步骤）

4.5 部署与维护

- 环境变量管理: 所有敏感配置(API Key、数据库路径)使用环境变量
- 日志记录: LangChain 自带 verbose 模式，建议生产环境使用 loguru
- 监控告警: API 调用失败率、平均响应时间、每日 Token 消耗
- Prompt 迭代: 定期 review Prompt 效果，根据 LLM 输出质量持续优化
- 向量库维护: 定期更新简历库和 JD 库的向量索引

5. 部署与使用

5.1 环境要求

- Python 3.10+
- Node.js 18+
- 4GB 以上可用内存（用于运行 Embedding 模型）

5.2 安装步骤

第一步：安装后端依赖

```
cd jobcopilot/backend  
pip install -r requirements.txt --break-system-packages
```

第二步：启动后端服务

```
cd jobcopilot/backend  
python main.py  
# 服务运行在 http://localhost:8000  
# API 文档: http://localhost:8000/docs
```

第三步：安装前端依赖

```
cd jobcopilot/frontend  
npm install
```

第四步：启动前端开发服务器

```
cd jobcopilot/frontend  
npm run dev  
# 服务运行在 http://localhost:5173
```

第五步：打开浏览器访问 <http://localhost:5173>

5.3 使用流程

推荐使用流程：

1. 打开"JD 分析"页 → 粘贴目标职位的 JD → 点击分析 → 获得结构化分析结果
2. 打开"简历优化"页 → 粘贴简历和 JD 分析结果 → 获取匹配度评分和优化建议
3. 点击"定向优化" → 查看 AI 改写后的简历 → 人工审阅调整
4. 打开"求职信"页 → 选择风格 → 一键生成求职信
5. 打开"投递管理"页 → 新增投递记录 → 追踪投递进度

5.4 注意事项

- API 配置: 项目默认使用 hongqiye 中转的 DeepSeek API，如需更换请在 backend/config.py 中修改
- 首次启动: 首次运行时会自动下载 BGE 中文嵌入模型(约 400MB)，请耐心等待
- 向量库: ChromaDB 数据存储在 backend/data/chroma_db/，如需重置可删除该目录
- AI 生成内容: 所有 AI 生成的内容(简历改写、求职信)建议人工审核后使用

6. 项目总结

6.1 技术亮点

1. Prompt 工程实践

- 三个核心模块各自使用不同的 Prompt 技术(Few-shot, Chain-of-Thought, Prompt Chain)
- 结构化的JSON 输出 + 容错处理，确保系统稳定性
- 角色设定增强 LLM 输出质量

2. RAG 检索增强

- ChromaDB 轻量级方案，无需外部服务
- 中文语义嵌入模型本地运行
- 双 Collection 设计(简历库 + JD 库)，支持交叉检索

3. Agent 架构设计

- 职业 ReAct 模式实现多步骤任务自动规划
- 双重调用模式平衡灵活性和效率
- 参考 how-claude-code-works 的 Agent Loop 设计理念

4. 工程化实践

- 前后端分离架构
- SQLAlchemy ORM + SQLite 数据持久化
- 完整的错误处理和用户反馈机制

6.2 与 AutoGEO 项目的技术延续

JobCopilot 延续了 AutoGEO 项目中积累的前端技术栈经验(Vue 3 + Element Plus + TypeScript)，同时在 AI 技术深度上更进一步：

- AutoGEO 中使用 n8n 作为 AI 调度 → JobCopilot 中自建 LangChain Agent 引擎
- AutoGEO 中 AI 功能通过 API 封装 → JobCopilot 中深入 Prompt 和 RAG 技术细节
- AutoGEO 重工程集成 → JobCopilot 重 AI 能力展示

两个项目形成了良好的互补关系，一个展示 AI 产品集成的广度，一个展示 AI 技术实现的深度。

6.3 学习收获

通过完成JobCopilot项目，可以深入理解以下AI核心技术：

1. Prompt Engineering: 从写"请帮我..."到设计 Few-shot 模板、CoT 推理链、JSON Schema 约束
2. RAG 全流程: Embedding→分块→存储→检索→注入 Prompt，理解每个环节的设计选择
3. Agent 原理: ReAct 循环、工具注册、思考链管理，从用 Agent 到懂 Agent
4. 全栈工程化: FastAPI + SQLAlchemy + Vue 3 + TypeScript 的完整前后端开发

这些技能对于 AI 产品经理岗位尤为关键 —— 既理解 AI 能力边界，又能和技术团队有效沟通。